Introduction to NS3

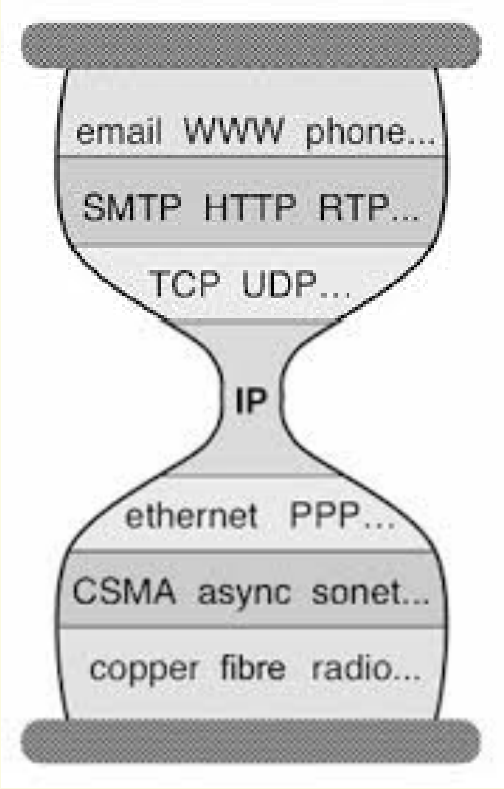
Dr. S. Amirtharaj

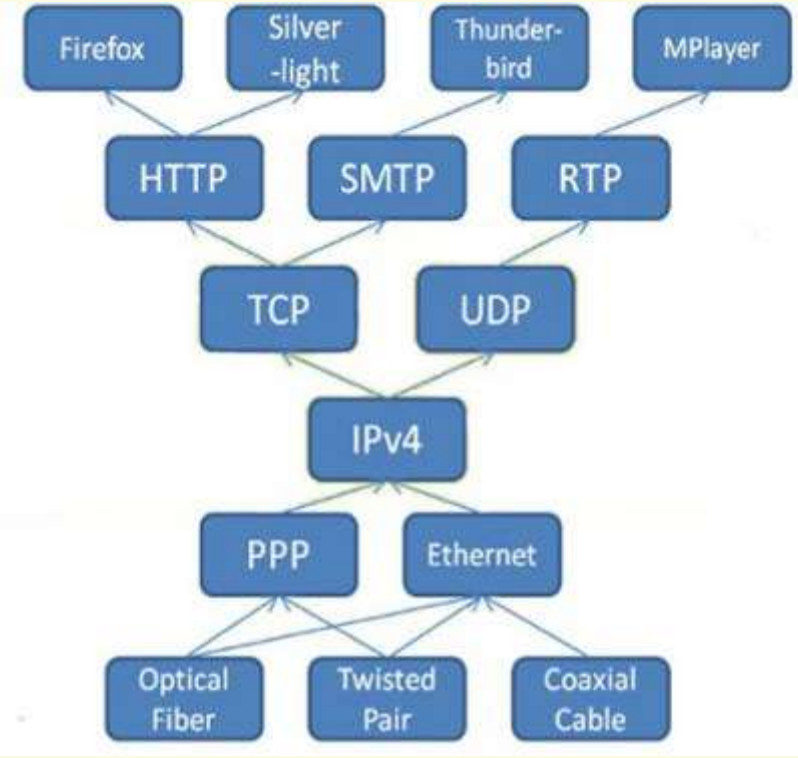
Department of Computer Applications

Mepco Schlenk Engineering College, Sivakasi

amrit@mepcoeng.ac.in

Network - The Hour Glass Model





What is ns-3?

ns-3 is a discrete-event network simulator for Internet systems

- **ns-3 allows researchers to study Internet protocols and large-scale systems in a controlled environment**
- **ns-3 is a new simulator (not backwards-compatible with ns-2)**

ns-3 is a free, open source software project organized around research community development and maintenance

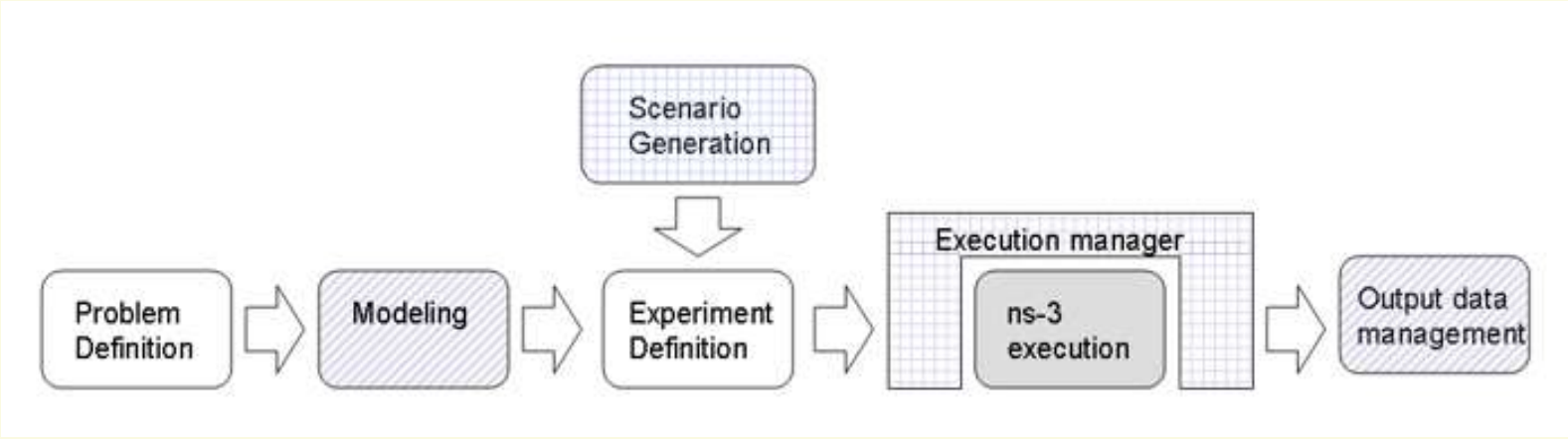
- **the target user community is networking researchers and educators**

What is ns3 …contd

- Written in C++ with optional Python interface and extensively documented API (doxygen)
- ns-3 is a GNU GPLv2-licensed project
- ns-3 is mainly supported for Linux, OS X, and FreeBSD, Windows Visual Studio port available

Discrete event network simulator

- Model of the evolution of a networked system through discrete events in time
- Used for experimentation and education



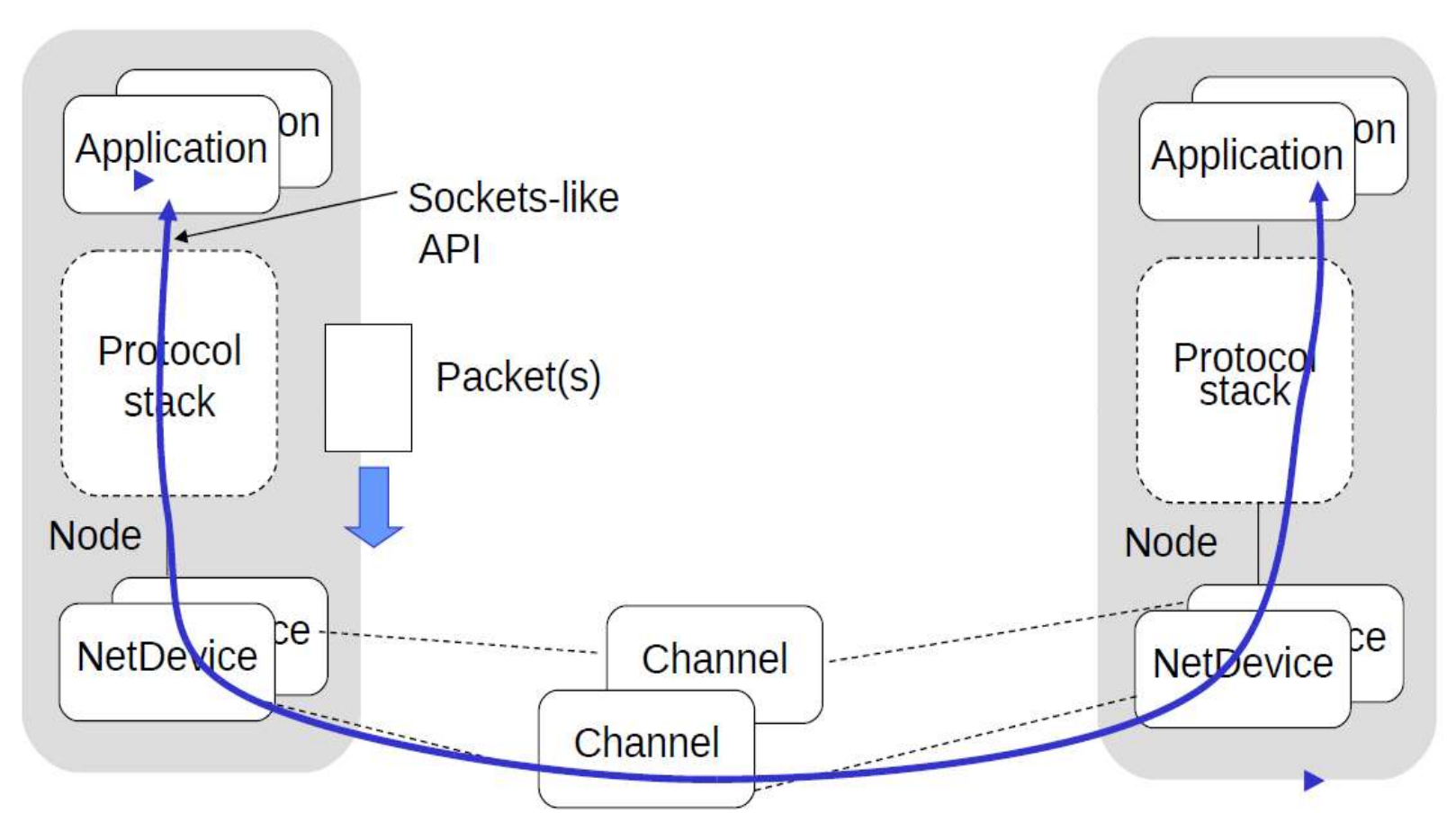
ns-3 simulation basics

- Simulation time advances in discrete jumps from event to event
- C++ functions schedule events to occur at specific simulation times
- A simulation scheduler orders the event execution
- **Simulation::Run()** gets it all started
- Simulation stops at specific time or when events end

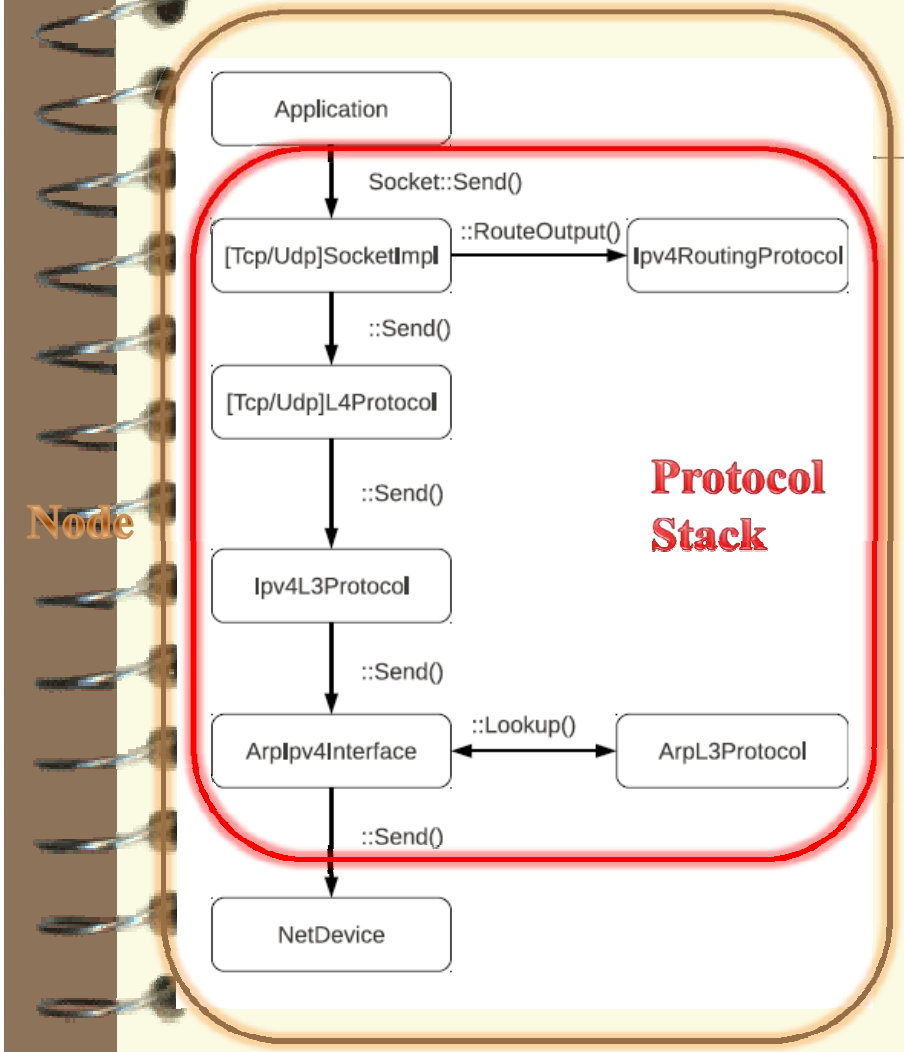
Abstractions

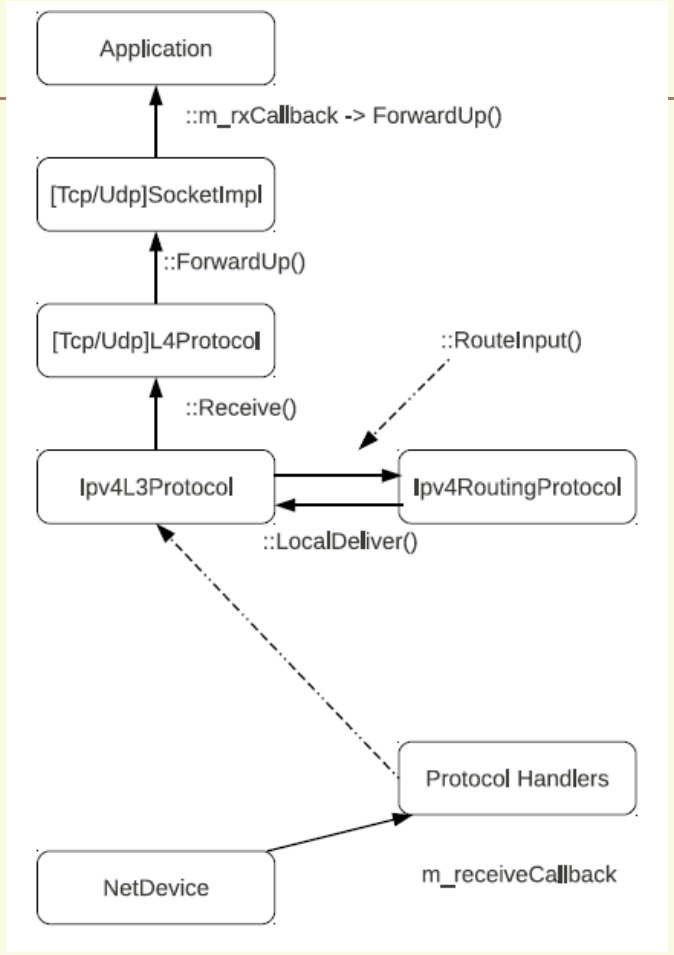
- Node
- Application
- Channel
- NetDevice
- Packet
- Topology Helpers – aggregate functionality of modules to make common operations easier than using the low-level API
- Consists of:
  - container objects
  - helper classes

Example - conceptual



Example – ns3 implemented



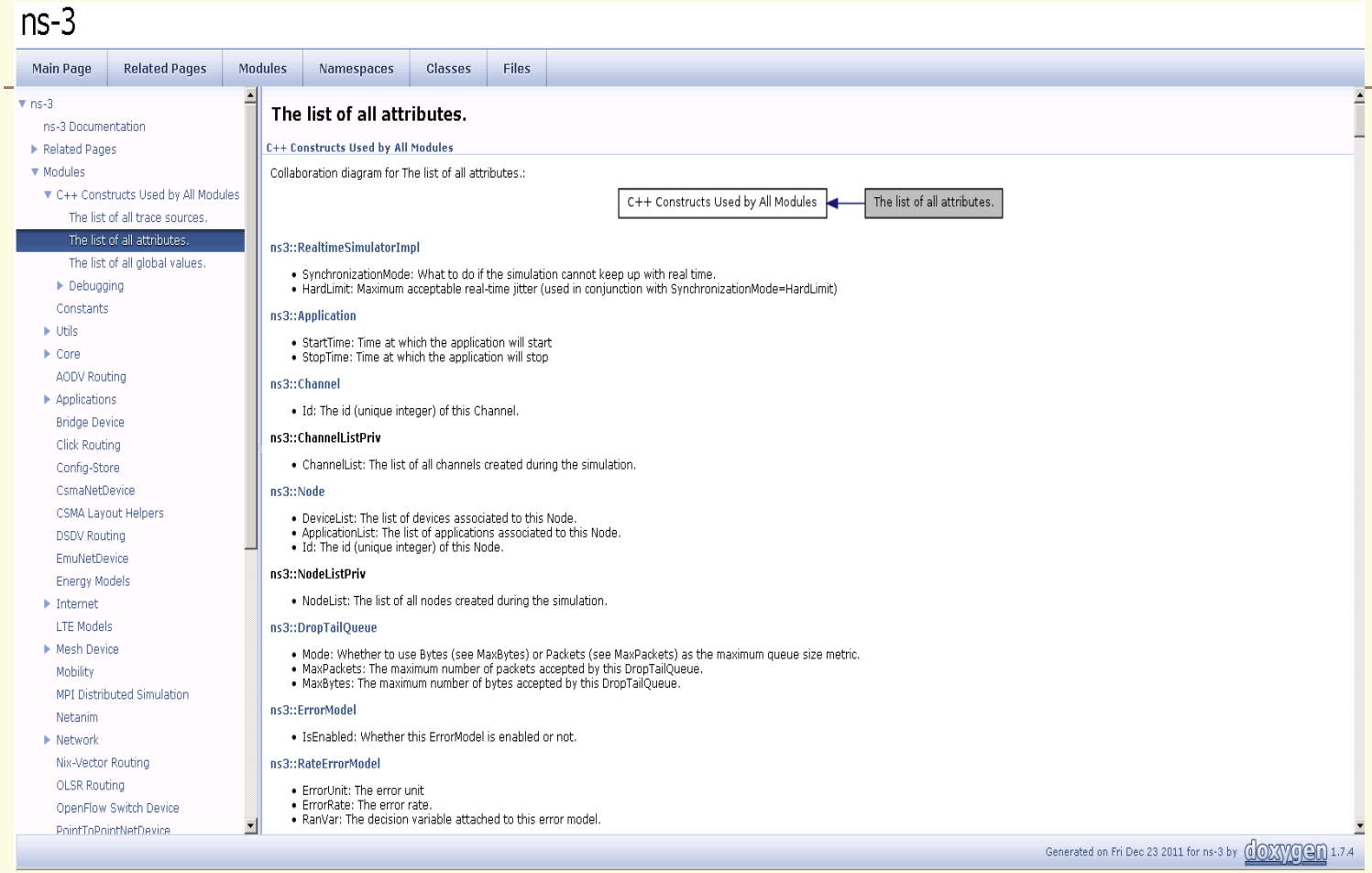


Attributes

Problem: Researchers want to identify all of the values affecting the results of their simulations and configure them easily

- ns-3 solution: Each ns-3 object has a set of attributes:
  - A name, help text
  - A type
  - An initial value
- Control all simulation parameters for static objects
- Dump and read them all in configuration files
- Visualize them in a GUI
- Makes it easy to verify the parameters of a simulation

Attributes…contd



http://www.nsnam.org/docs/release/3.13/doxygen/group___attribute_list.html

Attributes…contd

- An Attribute represents a value in our system
- An Attribute can be connected to an underlying variable or function

– e.g., TcpSocket::m_cwnd;
– or a trace source

How to handle attributes

The traditional C++ way:

- export attributes as part of a class's public API
- walk pointer chains (and iterators, when needed) to find what you need
- use static variables for defaults

The attribute system provides a more convenient API to the user to do these things

Navigating the attributes

- Attributes are exported into a string-based namespace, with filesystem-like paths
  - namespace supports regular expressions
- Attributes also can be used without the paths
  - e.g., “ns3::WifiPhy::TxGain”
- A Config class allows users to manipulate the attributes

Fine-grained attribute handling

- Set or get the current value of a variable
  - Here, one needs the path in the namespace to the right instance of the object

```
Config::SetAttribute("/NodeList/5/DeviceList/3
/Phy/TxGain", DoubleValue(1.0));
DoubleValue d;
nodePtr->GetAttribute
("/NodeList/5/NetDevice/3/Phy/TxGain", d);
```

- Users can get Ptrs to instances also, and Ptrs to trace sources, in the same way

How to manipulate attributes

- Individual object attributes often derive from default values
  - Setting the default value will affect all subsequently created objects
  - Ability to configure attributes on a per-object basis
- Set the default value of an attribute from the command-line:

```
CommandLine cmd;
cmd.Parse (argc, argv);
```

- Set the default value of an attribute with NS_ATTRIBUTE_DEFAULT
- Set the default value of an attribute in C++:

```
Config::SetDefault ("ns3::Ipv4L3Protocol::CalcChecksum",
BooleanValue (true));
```

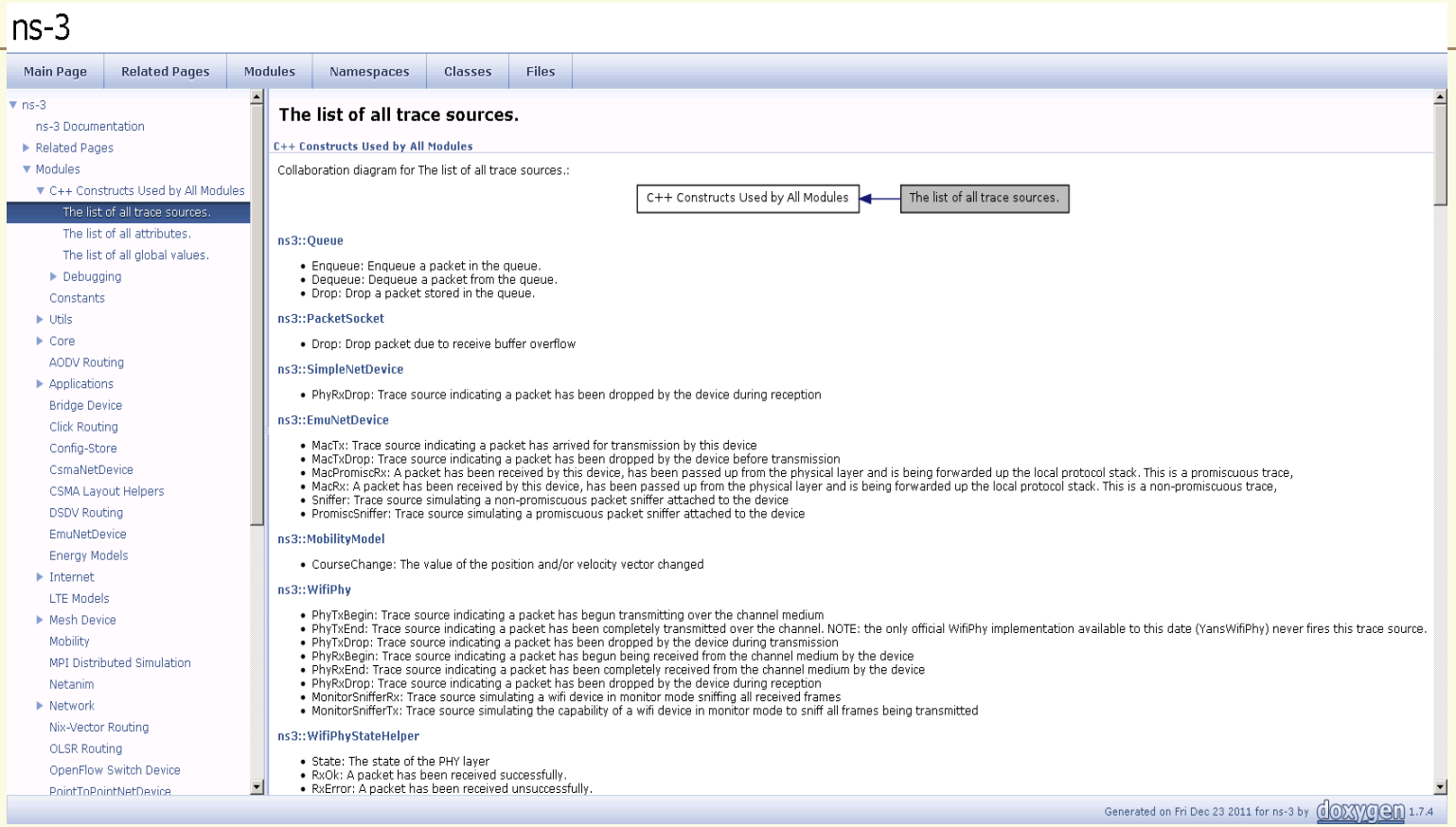
- Set an attribute directly on a specic object:

```
Ptr<CsmaChannel> csmaChannel = ...;
csmaChannel->SetAttribute ("DataRate",
StringValue ("5Mbps"));
```

Tracing System

- Simulator provides a set of pre-configured trace sources
  - Users may edit the core to add their own
- Users provide trace sinks and attach to the trace source
  - Simulator core provides a few examples for common cases
- Multiple trace sources can connect to a trace sink

Trace Sources



http://www.nsnam.org/docs/release/3.13/doxygen/group___trace_source_list.html

Basic Tracing

- Helper classes hide the tracing details from the user, for simple trace types
  - ascii or pcap traces of devices

```
std::ofstream ascii;
ascii.open ("wns3-helper.tr");
CsmaHelper::EnableAsciiAll (ascii);
CsmaHelper::EnablePcapAll ("wns3-
helper");
YansWifiPhyHelper::EnablePcapAll
("wsn3-helper");
```

Multiple Levels of Tracing

- Highest-level: Use built-in trace sources and sinks and hook a trace file to them
- Mid-level: Customize trace source/sink behaviour using the tracing namespace
- Low-level: Add trace sources to the tracing namespace

Highest Level of Tracing

- Highest-level: Use built-in trace sources and sinks and hook a trace file to them

```
// Also configure some tcpdump traces;
//each interface will be traced.
// The output files will be named:
// simple-point-to-point.pcap-<nodeId>-<interfaceId>
// and can be read by the "tcpdump -r" command (use
"-// tt" option to display timestamps correctly)

PcapTrace pcaptrace ("simple-point-to-point.pcap");
pcaptrace.TraceAllIp ();
```

Mid Level of Tracing

- Mid-level: Customize trace source/sink behaviour using the tracing namespace

```
void
PcapTrace::TraceAllIp (void)
{
  NodeList::Connect ("/nodes/*/ipv4/(tx|rx)",
  MakeCallback (&PcapTrace::LogIp, this));
}
```

Regular expression editing

Hook in a different trace sink

Low Level of Tracing

- Low-level: Add trace sources to the tracing namespace

```
Config::Connect ("/NodeList/.../Source",
MakeCallback (&ConfigTest::ChangeNotification,
this));
```

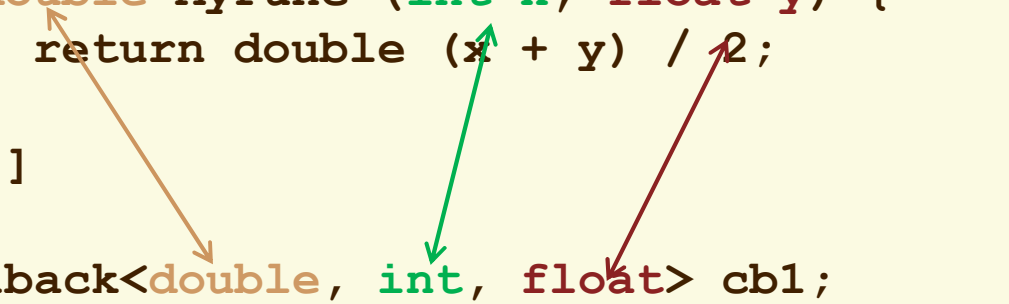
Callbacks

- ns-3 Callback class implements ***function objects***
  - Type safe callbacks, manipulated by value
  - Used for example in sockets and tracing

- Example

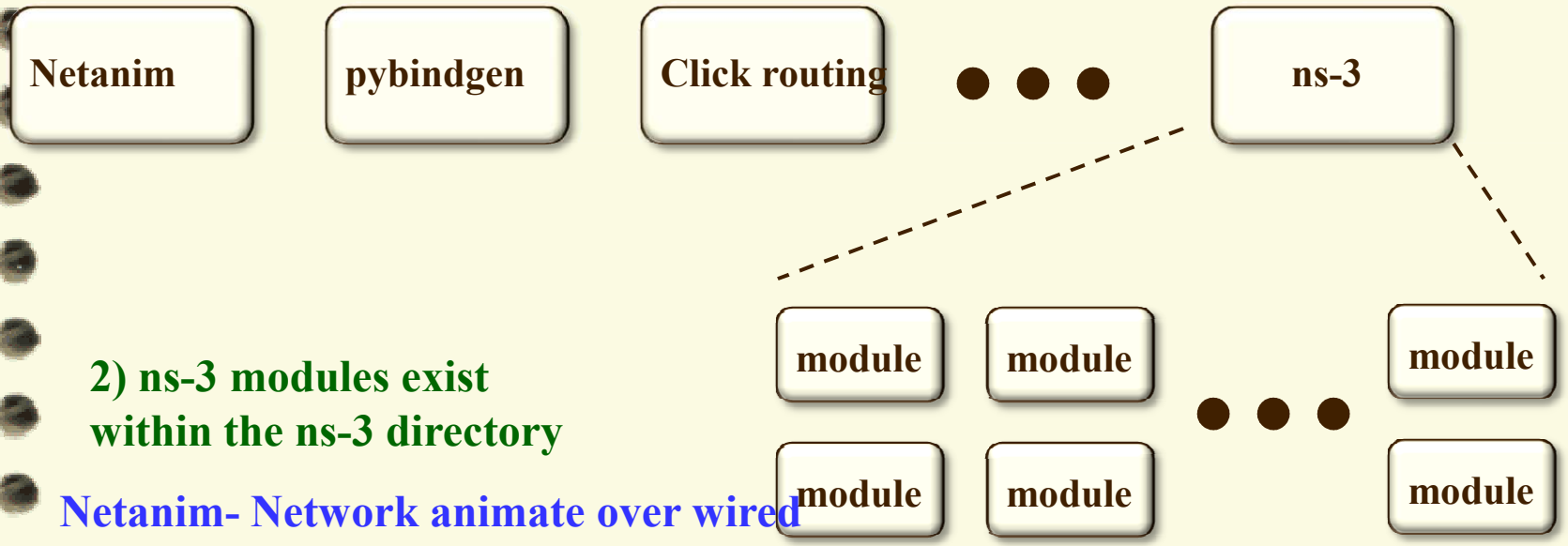
```
Class MyClass {
public:
    double MyFunc (int x, float y) {
      return double (x + y) / 2;
    }
[...]

Callback<double, int, float> cb1;
MyClass myobj;
cb1 = MakeCallback(&MyClass::MyFunc, &myobj);
double result = cb1 (2,3); // result receives 2.5
```



Software organization-Two levels of ns-3 software and libraries

1) Several supporting libraries, not system-installed, can be in parallel to ns-3



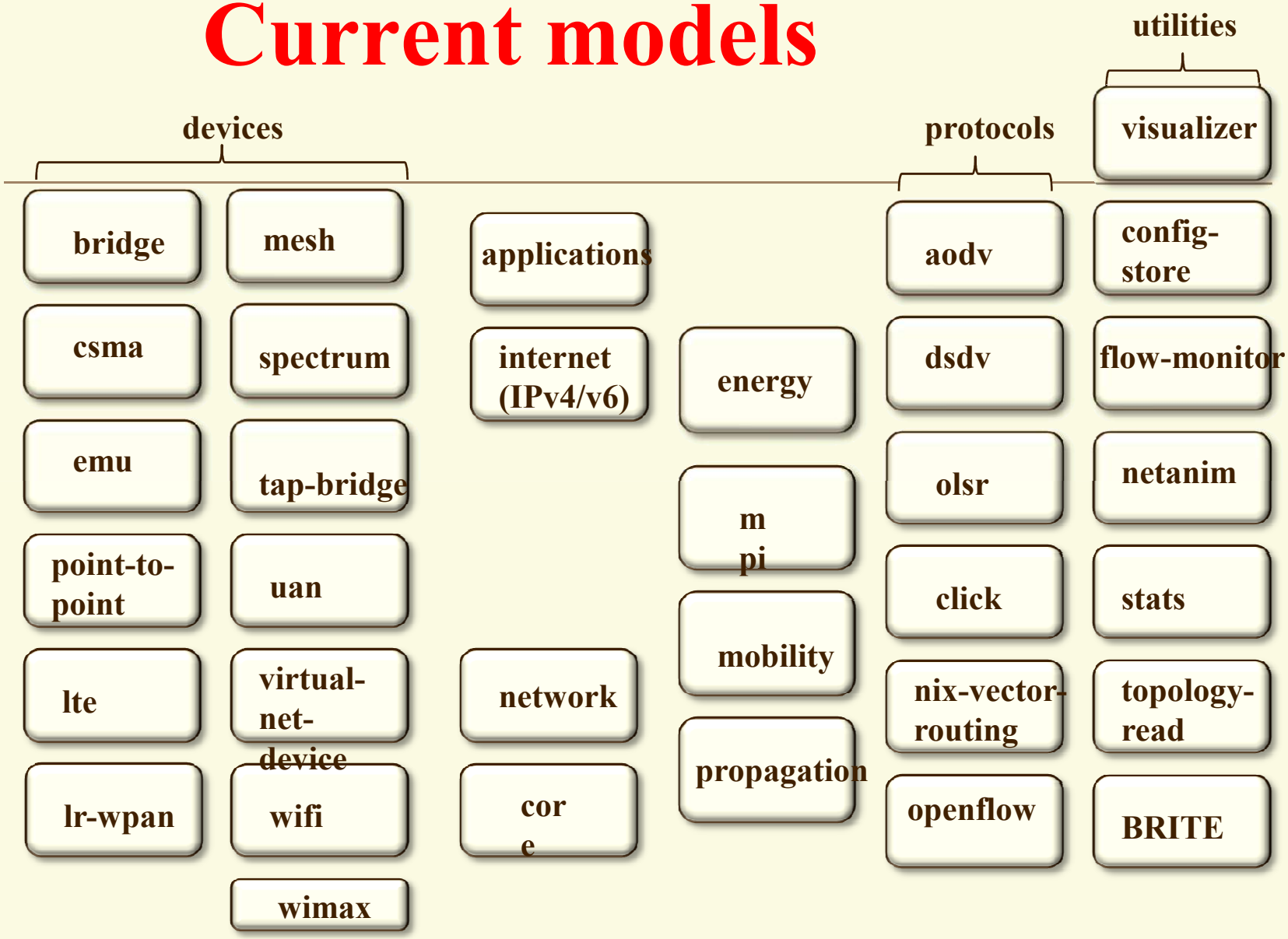
2) ns-3 modules exist within the ns-3 directory

Netanim- Network animate over wired and wireless link

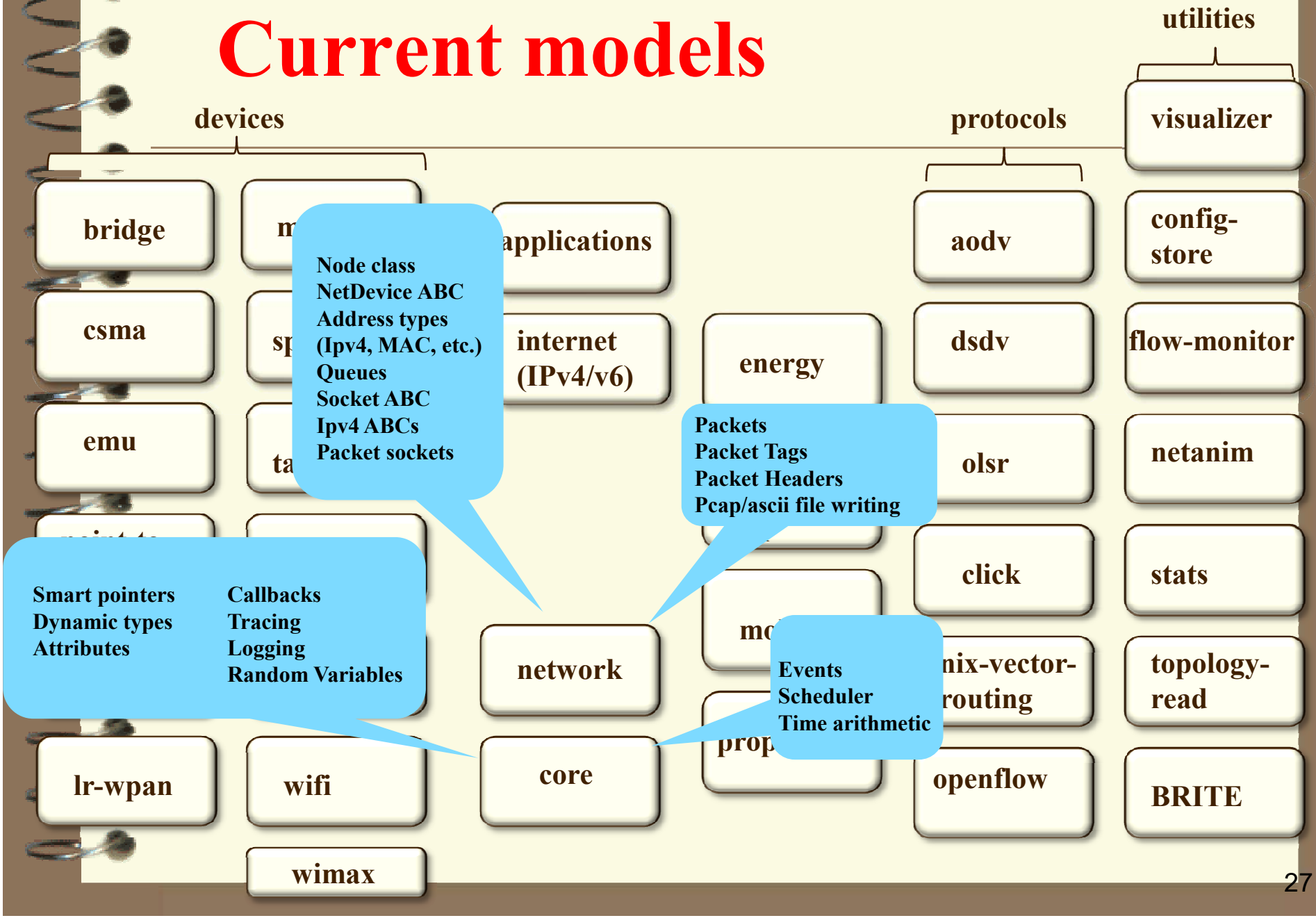
Pybindgen - A tool to generate Python bindings for C/C++ code

Click routing - Class to allow a node to use Click for external routing

Current models



Current models



Bridge	a virtual net device that bridges multiple LAN segments
Mesh	MAC layer mesh networking
Csma	Carrier sense multiple access
Emu	The `Emu` net device allows a simulation node to send and receive packets over a real network. The emulated net device relies on a specified interface being in promiscuous mode
Spectrum	The Spectrum module aims at providing support for modeling the frequency-dependent aspects of communications in *ns-3*
Tap-bridge	The Tap Bridge is designed to integrate "real" internet hosts (or more precisely, hosts that support Tun/Tap devices) into ns-3 simulations. Tun - The TUN is Virtual Point-to-Point network device Tap - The TAP is a Virtual Ethernet network device
Point-to-point	PP device

Uan	enable researchers to model a variety of underwater network scenarios (Channel, Physical, MAC)
LTE	LENA project model (LTE - Radio Model)
Lr-wpan	Low Rate wireless personal area network
Virtual-net device	A *virtual device*, similar to Linux TUN/TAP interfaces
Wifi	Wifi device
Wimax	Wimax
Energy	Energy related modules
MPI	Message Passing Interface
Mobility	Moving objects model
Propagation	Propagation model

Protocols

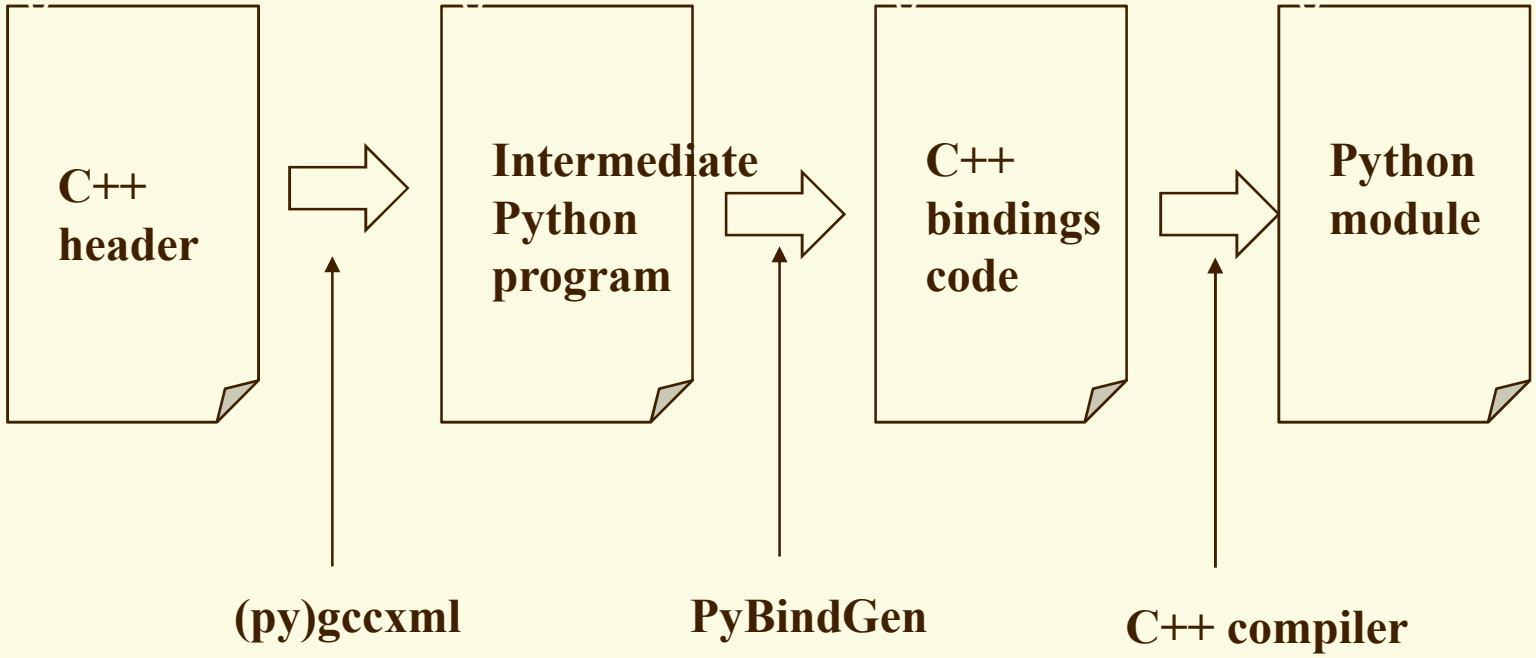
AODV	Adhoc on-demand distance vector protocol
Dsdv	Destination-Sequenced Distance-Vector Routing (Bellman Ford
Olsr	Optimistic Link state routing protocol
Click	Routing
Nix-vector routing	Nix-vector routing is a simulation specific routing protocol and is intended for large network topologies
openflow	To enhance ns3 with SDN

Utilities

visualizer	Visualizing the flow of ns3 simulation
BRITE	Topology generation tool

Python bindings

- ns-3 uses a program called PyBindGen to generate Python bindings for all libraries



Example Script - 1

```
/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */
// GPLv2 Licence …

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");

int main (int argc, char *argv[])
{
  LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
  LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);

  NodeContainer nodes;
  nodes.Create (2);

  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices;
  devices = pointToPoint.Install (nodes);
```

include modules that will be used

ns-3 project namespace

enable and disable console message logging by reference to the name

Topology Configuration

Example Script - 2

```
  InternetStackHelper stack;
  stack.Install (nodes);

  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");

  Ipv4InterfaceContainer interfaces = address.Assign (devices);

  UdpEchoServerHelper echoServer (9);

  ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));
  serverApps.Start (Seconds (1.0));
  serverApps.Stop (Seconds (10.0));

  UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
  echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
  echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
  echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

  ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));
  clientApps.Start (Seconds (2.0));
  clientApps.Stop (Seconds (10.0));

  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

Set up internet stack

Set up applications

Run the simulation

Running Example

ALL SCENARIOS SHOULD BE RUN UNDER SCRATCH

```
% cp examples/tutorial/first.cc scratch/myfirst.cc
% ./waf
% ./waf --run /scratch/myfirst
% Waf: Entering directory `/scratch/ns3-
workshop/ns-allinone-3.13/ns-3.13/build'
Waf: Leaving directory `/scratch/ns3-workshop/ns-
allinone-3.13/ns-3.13/build'
'build' finished successfully (1.218s)
Sent 1024 bytes to 10.1.1.2
Received 1024 bytes from 10.1.1.1
Received 1024 bytes from 10.1.1.2
```



http://www.nsnam.org/docs/ns-3-overview.ppt
http://www.nsnam.org/docs/ns-3-overview.pdf
http://www.nsnam.org/docs/release/3.13/tutorial/ns-3-tutorial.pdf
http://www.nsnam.org/docs/release/3.13/manual/ns-3-manual.pdf


THANK YOU